Table of Contents

I. INTRODUCTION 3
II. PRIMARY DATA TYPES AND OS ALLOCATION 3
A. Data Type Sizes Across Architectures 3
B. Practical Example: Observing Data Type Sizes 3
Summary 3
III. OS MEMORY SEGMENTS 4
A. Memory Segment Overview 4
B. Memory Layout Visualization 4
IV. PRACTICAL CODE EXAMPLES 5
A. Observing Variable Addresses 5
Summary 5
B. Pointer and Memory Allocation 5
Summary 5
V. LABORATORY EXERCISES 6
A. Lab 1: Data Type Size Exploration 6
1) C Program 6
2) Compile and Run 6
3) Comparing 6
4) Why might sizeof(long) differ between 32-bit and 64-bit systems? 6
B. Lab 2: Memory Segment Address Mapping 7
1) C Program 7
2) Compile and Run 7
3) Identify which segment has the highest addresses 7
4) Identify which segment has the lowest addresses 7
5) Calculate the address difference between stack and heap 7
Summary 7
VI. SUMMARY AND KEY TAKEAWAYS 8
VII. Knowledge Check Questions 8

Table of Figures

Figure 1: Table about Data Types 3
Figure 2: . Program to Observe Data Type Sizes 3
Figure 3: Table showing Memory Segments and their purposes 4
Figure 4: Process Memory Layout (High to Low Addresses) 4
Figure 5: Program to Observe Variable Addresses 5
Figure 6: Correct Pointer and Heap Usage 5
Figure 7: C program that prints the size 6
Figure 8: Compilation and Output 6
Figure 9: Program that declares variables in each segment 7
Figure 10: Compilation and Output 7

Lab 4: A Practical Approach to Understanding System Internals (Data Types and OS Memory Management)

I. INTRODUCTION

Data types form the foundation of programming and operating system interaction. It specifies:

1) The number of bytes to allocate in memory

2) How to interpret those bytes (integer, character, floating point)

3) What operations are valid on that data

When a programmer declares a variable, the OS must allocate physical or virtual memory to store that data. Understanding this relationship is crucial for writing efficient, secure, and portable code.

II. PRIMARY DATA TYPES AND OS ALLOCATION

A. Data Type Sizes Across Architectures

Table I shows how data type sizes vary across different system architectures, directly impacting OS memory allocation.

DATA TYPE SIZES ACROSS SYSTEM ARCHITECTURES

Data Type	16-bit	32-bit	64-bit	OS Impact
char	1 byte	1 byte	1 byte	Consistent across architectures
short	2 bytes	2 bytes	2 bytes	Memory-efficient counters
int	2 bytes	4 bytes	4 bytes	Affects pointer arithmetic
long	4 bytes	4 bytes	8 bytes	Critical for 64-bit addressing
long long	8 bytes	8 bytes	8 bytes	Large file offsets
float	4 bytes	4 bytes	4 bytes	IEEE 754 standard
double	8 bytes	8 bytes	8 bytes	Precision calculations
pointer	2 bytes	4 bytes	8 bytes	Address space limit

Figure 1: Table about Data Types

B. Practical Example: Observing Data Type Sizes

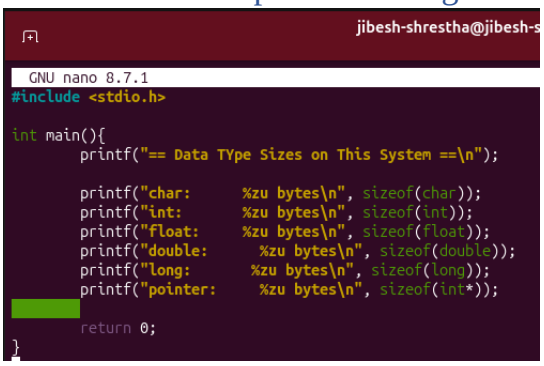


Figure 2: . Program to Observe Data Type Sizes

Summary

We wrote a code to observe the sizes of the data types in C.

III. OS MEMORY SEGMENTS

The operating system creates a process address space—a virtual view of memory divided into segments. Each segment serves a specific purpose and has different characteristics.

A. Memory Segment Overview

TABLE II
PROCESS MEMORY SEGMENTS AND THEIR PURPOSES

Segment	Purpose	Grows	Managed By
Text	Machine code (instructions)	Fixed	Compiler/Loader
Data	Initialized global/static variables	Fixed	Compiler/OS
BSS	Uninitialized global/static variables	Fixed	OS
Heap	Dynamically allocated memory	Upward	Programmer + OS
Stack	Local variables, function calls	Downward	Compiler + OS

Figure 3: Table showing Memory Segments and their purposes

B. Memory Layout Visualization

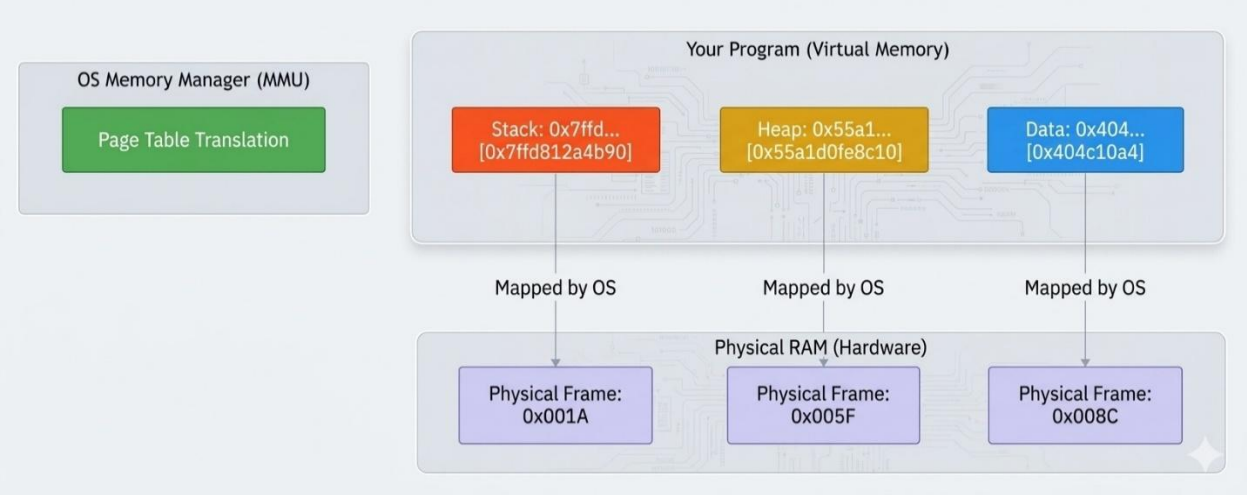


Figure 4: Process Memory Layout (High to Low Addresses)

Key Observation: Stack addresses are high (0x7ffd...),

Heap addresses are mid-range (0x55a1...), and

Data/BSS addresses are low (0x404...).

IV. PRACTICAL CODE EXAMPLES

A. Observing Variable Addresses



Figure 5: Program to Observe Variable Addresses

Summary

This C program shows how a computer's memory is organized into different memory segments (Data Segment, BSS, Stack, and Heap) based on variables are declared.

B. Pointer and Memory Allocation



Figure 6: Correct Pointer and Heap Usage

Summary

This program demonstrates how a pointer variable on the Stack holds the memory address of a value stored on the Heap**.**

V. LABORATORY EXERCISES

A. Lab 1: Data Type Size Exploration

1) C Program

```
GNU nano 8.7.1
#include <stdio.h>
int main(){
        printf("== Data TYpe Sizes on This System ==\n");

        printf("char:      %zu bytes\n", sizeof(char));
        printf("int:       %zu bytes\n", sizeof(int));
        printf("float:     %zu bytes\n", sizeof(float));
        printf("double:      %zu bytes\n", sizeof(double));
        printf("long:       %zu bytes\n", sizeof(long));
        printf("unsigned int:     %zu bytes\n", sizeof(unsigned int));
        printf("long long:         %zu bytes\n", sizeof(long long));

        return 0;
}
```

Figure 7: C program that prints the size

2) Compile and Run

```
jibesh-shrestha@jibesh-shrestha-VirtualBox:~/Batch38$ gcc datatype.c -o datatype
jibesh-shrestha@jibesh-shrestha-VirtualBox:~/Batch38$ ./datatype
== Data TYpe Sizes on This System ==
char:      1 bytes
int:       4 bytes
float:     4 bytes
double:      8 bytes
long:       8 bytes
unsigned int:     4 bytes
long long:         8 bytes
jibesh-shrestha@jibesh-shrestha-VirtualBox:~/Batch38$
```

Figure 8: Compilation and Output

3) Comparing

The output is same with the 64 bit data type sizes

4) Why might sizeof(long) differ between 32-bit and 64-bit systems?

sizeof(long) can be different because 32-bit and 64-bit systems use different amounts of memory.

B. Lab 2: Memory Segment Address Mapping

1) C Program

```c
#include <stdio.h>
#include <stdlib.h>

int global_var= 150;      //Data Segment
int bss_var;     //BSS Segment
int main(){
        int local_var=30;               //Stack
        static int local_static=100;
        int *heap_var = (int *)malloc(sizeof(int));   //Heap
        *heap_var=500;
        printf("....Variable Addresses....\n\n");
        printf("global_var;       %p (Data)\n", (void *)&global_var);
        printf("bss_var:          %p (BSS)\n", (void *)&bss_var);
        printf("local_var;       %p (Stack)\n", (void *)&local_var);
        printf("local_static:     %p (Data)\n",(void *)&local_static);
        printf("heap_var ptr:     %p (Stack- Pointer)\n",(void *)&heap_var);
        printf("heap_var:          %p (Heap)\n", (void*)heap_var);

        long difference = (long)&local_var - (long)heap_var;
        if(difference < 0)
                difference = -difference;

        printf("\nAddress difference between Stack and Heap: %ld bytes\n",difference);
        free(heap_var);
        return 0;
}
```

Figure 9: Program that declares variables in each segment

2) Compile and Run

```
jibesh-shrestha@jibesh-shrestha-VirtualBox:~/Batch38$ gcc varadd.c -o varadd
jibesh-shrestha@jibesh-shrestha-VirtualBox:~/Batch38$ ./varadd
....Variable Addresses....

global_var;       0x64d8dc06b010 (Data)
bss_var:          0x64d8dc06b01c (BSS)
local_var;       0x7ffde3c20a64 (Stack)
local_static:     0x64d8dc06b014 (Data)
heap_var ptr:     0x7ffde3c20a68 (Stack- Pointer)
heap_var:          0x64d90b91b010 (Heap)

 Address difference between Stack and Heap: 29845059820116 bytes
jibesh-shrestha@jibesh-shrestha-VirtualBox:~/Batch38$
```

Figure 10: Compilation and Output

3) Identify which segment has the highest addresses

The Stack segment has the highest addresses (0x7ffde3c20a64).

4) Identify which segment has the lowest addresses

The Data segment has the lowest addresses (0x64d8dc06b010).

5) Calculate the address difference between stack and heap

The address difference between stack and heap is 29,845,059,820,116 bytes.

Summary

This is a C program with one variable in each part of memory and their addresses.

- **Data** (global with a value) → lowest address
- **BSS** (global with no value) → just after Data
- **Heap** (from malloc) → after BSS
- **Stack** (local variable) → highest address

VI. SUMMARY AND KEY TAKEAWAYS

Data types are contracts—they tell the OS how much memory to allocate and how to interpret it.

• Memory is segmented—stack for locals, heap for dynamic, data/BSS for globals, text for code.

• The OS manages it all—tracking, allocating, protecting, and virtualizing memory.

• Stack is automatic; heap is manual—choose based on lifetime and size needs.

• Virtual memory provides isolation—each process has its own address space.

• Pointers bridge stack and heap—the pointer variable lives on stack, but points to heap memory

VII. Knowledge Check Questions

1) What is a data type in the context of programming and operating systems?

It tells the type of value a variable hold (like number or letter) and how much memory it needs.

2) Name the five main memory segments in a process memory layout.

Text, Data, BSS, Heap, Stack.

3) What is the difference between the Data segment and the BSS segment?

Data stores variables with a starting value; BSS stores variables with no starting value (set to 0).

4) Why does the OS place the stack at high virtual ad dresses and the text segment at low addresses?

The text segment is allocated at the lowest virtual addresses because it is immutable (fixed in size). The stack segment is allocated at the highest virtual addresses because it is dynamic.

5) What happens to stack memory when a function returns? who reclaims it?

It is automatically freed by the OS.

6) How does the OS know which memory belongs to which process?

Each process has its own memory space, tracked by the OS using page tables.